

Literature Review: gPPM - A Generalized Matrix Operation and Parallel Algorithm for Erasure Code Encoding/Decoding

1. Background and Motivation

Erasure codes are used in large storage systems (cloud storage, data centers) to protect against disk, sector, and node failures while using less storage overhead than simple replication. The paper divides erasure codes into two families:

- Symmetric Parity codes (e.g., Reed-Solomon/RS, Cauchy RS, STAR) - every parity block is computed from the same number (k) of data blocks.
- Asymmetric Parity codes (e.g., SD code, PMDS code, LRC code) - different parity blocks are computed from different numbers of data blocks (e.g., local parity vs global parity in LRC, or disk parity vs sector parity in SD code). These newer codes were designed to handle real failure patterns better (like combined disk + sector failures, or fast degraded reads), but existing implementations still decode them using the traditional parity-check matrix method, which is slow, serial, and treats all lost data as one big block to recover together.

The authors' key observation: some lost blocks are actually independent of each other, they can be recovered separately, in parallel, without waiting on other lost blocks. Existing decoding methods miss this and process everything serially.

2. Contribution Overview

The paper proposes two algorithms:

- PPM (Partitioned and Parallel Matrix) - works only on asymmetric parity codes. It splits the parity-check matrix H into smaller sub-matrices based on which lost blocks are independent, decodes the independent parts in parallel across threads, and picks a cheaper calculation sequence (order of matrix multiplication) to reduce the number of operations.
- gPPM (generalized PPM) - extends PPM's idea to also work on symmetric codes like RS, by dynamically choosing, per-matrix, whichever calculation order (normal vs. matrix-first) needs fewer actual multiply/XOR operations. This makes it more broadly applicable and squeezes out further gains even inside PPM's sub-matrices.

This is a journal extension of an earlier ICPP 2015 conference paper (PPM only); gPPM is the new material.

3. Methodology

Step 1 - Cost model

The authors count operations in terms of `mult_XORs()` (multiply a block by a Galois-Field constant, then XOR into an accumulator). Given the decoding equation $BF = F^{-1} \cdot S \cdot BS$, there are two ways to compute it:

- Normal sequence: multiply $S \cdot BS$ first, then apply F^{-1} .
- Matrix_first sequence: multiply $F^{-1} \cdot S$ first (a small matrix×matrix product), then apply to BS .

Which is cheaper depends on the number of nonzero elements produced by each approach - this varies by code and parameters, so no single sequence always wins.

Step 2 - Exploit independence (PPM)

Using a log table derived from the parity-check matrix, the algorithm finds which lost blocks depend only on themselves (or a small self-contained group) versus which are entangled with everything else. Independent groups become separate small sub-matrices; the remaining sub-matrix handles the entangled dependent blocks.

Step 3 - Parallelize

The independent sub-matrices are decoded concurrently across up to T threads (T capped by core count to avoid overhead), then the recovered values feed into decoding the remaining sub-matrix.

Step 4 - Fine-grained sequence selection (gPPM)

Rather than statically picking one calculation order for the whole matrix (as PPM does), gPPM computes the cost of both sequences per sub-matrix (accounting separately for the relative cost of multiply vs. add operations) and picks the cheaper one on the fly. This also lets it apply the same trick to RS/GRS codes, which have no independent block structure to exploit via partitioning, only sequence choice.

Evaluation setup

Implemented in C on top of an open-source SD encoder/decoder, using Intel SIMD (ISA-L-style) acceleration, tested across three CPUs (4/6/8 cores), varying stripe size, number of disks (n), data/parity counts (k , m), sector counts (r , s , z), and thread count (T).

4. Key Results

- PPM improves SD/LRC code encode/decode speed by up to 210.81% (avg ~61% for SD, 16–37% for LRC).
- gPPM improves further over PPM by up to 102.41% in about a third of tested SD configurations (PPM remains optimal in the rest).
- gPPM improves plain RS code by up to 32.25%, and the RS variant used inside SD code (GRS) by up to 79.73%.
- Improvements hold across different CPUs, thread counts, and stripe sizes, confirming the gains come from reduced computation, not just hardware-specific tuning.
- Power/energy overhead from added threads is small (~2 W).

5. Related Work Positioning

The authors distinguish their approach from:

- New code designs (EVENODD, RDP, STAR, SD, LRC, EC-wide) - these change what is computed, not how it's computed.
- SIMD/GPU acceleration (Intel ISA-L, GPU-based decoding) - these speed up individual operations, and are complementary (used underneath PPM/gPPM in the experiments).
- APCM - also reduces computational cost via matrix transformation, but optimizes matrix-matrix operations (cheap) rather than matrix-vector operations against actual data blocks (expensive), so its real-world gains are limited.
- Other parallelism schemes (block-level, disk-level, node-level, equation-oriented) - PPM/gPPM is described as a matrix-oriented parallelism approach, distinct from these because it exploits the algebraic structure of independent faulty blocks in H itself.

6. Conclusion of the Paper

PPM/gPPM together give a general, code-agnostic technique to speed up erasure code encoding/decoding by combining (a) exploiting parallelism among independent lost blocks and (b) dynamically choosing the cheaper matrix computation order - applicable to both older symmetric codes (RS) and newer asymmetric codes (SD, LRC, PMDS).

Full Reference List (56 references from the paper)

1. Ultra-Low Latency with Samsung Z-NAND SSD.
2. Amazon EC2.
3. Intel Intelligent Storage Acceleration Library (Intel ISA-L).
4. Intel Optane SSD DC P5800X Series.
5. Optimizing Software for the Next Gen Intel Optane SSD P5800X.
6. H. P. Anvin. 2011. The mathematics of RAID-6.
7. Bairavasundaram, Goodson, Pasupathy, Schindler. 2007. An analysis of latent sector errors in disk drives (SIGMETRICS).
8. Bairavasundaram, Goodson, Schroeder, Arpaci-Dusseau, Arpaci-Dusseau. 2008. An analysis of data corruption in the storage stack (FAST).
9. Blaum, Brady, Bruck, Menon. 1995. EVENODD: An efficient scheme for tolerating double disk failures (IEEE Trans. Computers).
10. Blaum, Hafner, Hetzler. 2012. Partial-MDS Codes and their Application to RAID Type Architectures.
11. Blomer, Kalfane, Karp, Karpinski, Luby, Zuckerman. 1995. An XOR-Based Erasure-Resilient Coding Scheme.
12. Calder, Wang, Ogun, Nilakantan et al. 2011. Windows Azure Storage (SOSP).
13. Corbett, English, Goel, Grcanac, Kleiman, Leong, Sankar. 2004. Row-diagonal parity for double disk failure correction (FAST).
14. Ford, Labelle, Popovici, Stokely, Truong, Barroso, Grimes, Quinlan. 2010. Availability in globally distributed storage systems (OSDI).
15. Gu, Wu, Xie, Qiu, Li, Guo, He, Dong, Zhao. 2019. Optimizing the parity check matrix for efficient decoding of RS-based cloud storage (IPDPS).
16. Haddock, Bangalore, Curry, Skjellum. 2019. High performance erasure coding for very large stripe sizes (SpringSim).
17. Holland, Gibson, Siewiorek. 1993. Fast, on-line failure recovery in redundant disk arrays.
18. Hou, Menon, Patt. 1993. Balancing I/O response time and disk rebuild time in a RAID5 disk array.
19. Hu, Chen, Lee, Tang. 2012. NCCloud: Applying network coding for storage repair in a cloud-of-clouds (FAST).
20. Hu, Cheng, Yao, Lee, Wang, Chen. 2021. Exploiting combined locality for Wide-Stripe erasure coding (FAST).
21. Huang, Simitci, Xu, Ogun, Calder, Gopalan, Li, Yekhanin. 2012. Erasure coding in Windows Azure Storage (USENIX ATC).
22. Huang, Xu. 2005. STAR: An efficient coding scheme for correcting triple storage node failures (FAST).
23. Iliadis, Haas, Hu, Eleftheriou. 2008. Disk scrubbing versus intra-disk redundancy (SIGMETRICS).
24. Jiang, Hu, Zhou, Kanevsky. 2008. Are disks the dominant contributor for storage failures? (FAST).
25. Khan, Burns, Plank, Pierce, Huang. 2012. Rethinking erasure codes for cloud file systems (FAST).
26. Kim, Seshadri, Dickey, Chiu. 2014. Evaluating phase change memory for enterprise storage systems (FAST).
27. Lee, Ipek, Mutlu, Burger. 2009. Architecting phase change memory as a scalable DRAM alternative (ISCA).

- 28.Lee, Lui. 2002. Automatic recovery from disk failure in continuous-media servers (IEEE TPDS).
- 29.Li, Lee. 2014. STAIR codes: A general family of erasure codes for tolerating device and sector failures (FAST).
- 30.Li, Li, Lee, Huang. 2017. Repair pipelining for erasure-coded storage (USENIX ATC).
- 31.Li, Cao, Wan, Zhang, Xie, He, Subedi. 2015. PPM: A partitioned and parallel matrix algorithm to accelerate encoding/decoding of asymmetric parity erasure codes (ICPP) — the earlier conference version of this paper.
- 32.Li, Wang, Tian, Wu, Zhang, Xu. 2018. A hierarchical RAID architecture towards fast recovery and high reliability (IEEE TPDS).
- 33.Lin, Gong, Shen, Lee, Shu. 2021. Boosting full-node repair in erasure-coded storage (USENIX ATC).
- 34.MacWilliams, Sloane. 1977. The theory of error-correcting codes, Part I.
- 35.Menon, Mattson. 1992. Distributed sparing in disk arrays (COMPCON).
- 36.Mitra, Panta, Ra, Bagchi. 2016. Partial-Parallel-Repair (PPR) (EuroSys).
- 37.Peterson, Weldon. 1972. Error-Correcting Codes (2nd ed.).
- 38.Pinheiro, Weber, Barroso. 2007. Failure trends in a large disk drive population (FAST).
- 39.Plank. 2008. The RAID-6 liberation codes (FAST).
- 40.Plank. 2013. Open source encoder and decoder for SD erasure codes (Tech Report).
- 41.Plank, Blaum, Hafner. 2013. SD codes: Erasure codes designed for how storage systems really fail (FAST).
- 42.Plank, Greenan, Miller. 2013. Screaming fast Galois field arithmetic using Intel SIMD instructions (FAST).
- 43.Plank, Luo, Schuman, Xu, Wilcox-O'Hearn. 2009. A performance evaluation and examination of open-source erasure coding libraries (FAST).
- 44.Qureshi, Srinivasan, Rivers. 2009. Scalable high performance main memory system using phase-change memory technology (ISCA).
- 45.RANDOM.ORG. 2022. Random integer generator.
- 46.Reed, Solomon. 1960. Polynomial codes over certain finite fields (J. SIAM).
- 47.Sathiamoorthy, Asteris, Papailiopoulos, Dimakis, Vadali, Chen, Borthakur. 2013. XORing elephants: Novel erasure codes for big data (VLDB).
- 48.Schroeder, Damouras, Gill. 2010. Understanding latent sector errors and how to protect against them (FAST).
- 49.Schroeder, Gibson. 2007. Disk failures in the real world (FAST).
- 50.Sobe. 2010. Parallel Reed/Solomon coding on multicore processors.
- 51.Tian, Feng, Jiang, Zeng, Chen, Wang, Song. 2007. PRO: A popularity-based multi-threaded reconstruction optimization for RAID (FAST).
- 52.Wan, Wang, Xie, Yang. 2014. S2-RAID: Parallel RAID architecture for fast data recovery (IEEE TPDS).
- 53.Wan, He, Huang, Cao, Li, Xie. 2013. An efficient penalty aware cache for parity-based disk arrays under faulty conditions (IEEE TPDS).
- 54.Wu, Du, Lee, Li, Xu. 2021. StripeMerge: Efficient wide-stripe generation for large-scale erasure-coded storage (ICDCS).
- 55.Yao, Hu, Cheng, Lee, Feng, Wang, Chen. 2022. Optimal data placement for stripe merging in locally repairable codes (INFOCOM).
- 56.Zhong, Li, Wu, Zarkadas, Tao, Mesterhazy, Makris, Yang, Tai, Stutsman, Cidon. 2022. XRP: In-kernel storage functions with eBPF (OSDI).

Papers Citing This Paper

Chen, J., Yang, S., Wang, Y., Ye, M., Lei, F. (2024) - Data repair accelerating scheme for erasure-coded storage system based on FPGA and hierarchical parallel decoding structure, Cluster Computing, 27(6): 7803–7823. The

cited paper uses this paper's check-matrix-based decoding approach as part of an FPGA hardware acceleration design for Cauchy Reed-Solomon (CRS) code repair.

7. Sequential Baseline Implementation and Results

7.1 Implementation Scope

A sequential C reference implementation of the traditional SD-code encoding/decoding process from Section 2.2 of the gPPM paper was developed. The implementation constructs the parity-check matrix H , separates it into faulty and surviving matrices F and S , computes the inverse of F over $GF(2^w)$, and reconstructs the faulty blocks using $BF = F^{-1} \times (S \times BS)$.

This baseline is intentionally single-threaded, scalar, and limited to the normal matrix calculation sequence. It contains no PPM partitioning, OpenMP, SIMD, or gPPM sequence selection. $GF(2^8)$, $GF(2^{16})$, and $GF(2^{32})$ are supported, allowing every configuration in the published FAST SD coefficient table to be evaluated.

7.2 Correctness Validation

Correctness was checked in two stages. First, the SD(4,4,1,1) worked example from Figure 2 was reproduced with 4 KiB sectors. It produced $u(S) = 22$, $u(F^{-1}) = 13$, and $C = 35$, exactly matching $C1 = 35$. Encoding and recovery both satisfied $H \times B = 0$, and the recovered stripe was byte-identical to the original encoded stripe.

Second, the implementation was driven by all 3,969 published SD coefficient sets. Combining each configuration with every defined value $1 \leq z \leq s$ produced 7,938 parameter points. Of these, 105 are geometrically impossible because $s > z(n-m)$, leaving 7,833 feasible points. For each feasible point, the sweep tests one deterministic representative layout: the leftmost m disks and s additional sectors distributed over exactly z rows. It covers the full parameter grid, not every combinatorial fault placement.

Validation measure	Result
Published coefficient configurations	3,969
Total configuration $\times$ z points	7,938
Geometrically infeasible points	105
Representative feasible layouts	7,833
Encode validation passes	7,833 / 7,833
Decode round-trip passes	7,833 / 7,833
Singular recovery matrices	0
$C = u(S) + u(F^{-1})$ mismatches	0
Measured C greater than $C1$	0
Exact $C1$ matches	7,649 (97.65%)
Measured C below $C1$	184 (2.35%)

The 184 under-counts are not decoding failures. In every tested layout $u(S)$ matches the structural formula exactly, while an algebraic cancellation makes one or more entries of F^{-1} equal to zero. Consequently, the measured operation count is lower than the paper's generic-position $C1$ expression. There were no over-counts, count-identity errors, singular matrices, or data-recovery failures in the tested layouts.

7.3 Baseline Benchmark Method

The baseline was benchmarked on an Intel Core Ultra 9 185H system using GCC 11.4.0 with $-O2$. Although the machine exposes 22 logical CPUs, the benchmark uses one thread. The published paper used different processors and SIMD acceleration, so these absolute values are a local scalar baseline rather than a direct reproduction of the paper's throughput figures.

The compact suite fixes $n = 16$, $r = 16$, and $z = 1$, and tests every pair m, s in $\{1, 2, 3\}$. Each codeword is exactly 32 MiB (33,554,432 bytes), giving 256 sectors of 131,072 bytes each. The nine configurations naturally cover $GF(2^8)$, $GF(2^{16})$, and $GF(2^{32})$. One untimed warm-up is followed by ten measured encoding trials and ten measured decoding trials. Random data generation, erasure injection, correctness checks, checksums, and CSV output are outside the timed interval.

Throughput is reported using useful non-parity bytes divided by elapsed time, matching the convention in the upstream SD implementation. Full-codeword throughput and every raw trial are also retained in `benchmark_results.csv`.

7.4 Baseline Throughput Results

m	s	Field	C	Encode MiB/s	Decode MiB/s
1	1	$GF(2^8)$	527	852.60	854.58
1	2	$GF(2^{16})$	783	634.68	631.51
1	3	$GF(2^{32})$	1039	673.01	671.77
2	1	$GF(2^8)$	828	444.98	447.81

2	2	GF(2 ¹⁶)	1084	399.48	401.21
2	3	GF(2 ³²)	1340	479.27	476.19
3	1	GF(2 ⁸)	1159	282.56	275.55
3	2	GF(2 ¹⁶)	1415	276.56	279.68
3	3	GF(2 ³²)	1671	352.91	349.51

All 180 measured trials passed byte-for-byte recovery, syndrome, and operation-count validation. A checksum was also recorded for every trial in `benchmark_results.csv`. Throughput generally decreases as *m* and *s* increase because more parity equations and nonzero matrix coefficients must be applied to each stripe. Field width alone does not determine speed: the GF(2⁸), GF(2¹⁶), and GF(2³²) backends use different scalar table organizations, so their absolute rates should not be interpreted as a monotonic field-size trend.

These results establish the reference point for later milestones. The parallel PPM implementation must reproduce the same recovered bytes and $H \times B = 0$ invariant, while its speedup is calculated against the corresponding sequential throughput above.

7.5 Reproducibility

The implementation and reported results can be reproduced with:

- `make test`
- `make run`
- `make sweep`
- `make benchmark`

The complete per-point correctness data is stored in `sweep_results.csv`. The complete per-trial timing data, both throughput definitions, operation counts, and checksums are stored in `benchmark_results.csv`.